Pure Functional Programming with Haskell

Concepts of Programming Languages

Sebastian Macke
Rosenheim Technical University

Last lecture

- Functional programming in general
- Impure Functional programming in Go
- Anonymous functions
- Closures
- Currying
- Lambda Calculus
- Streaming functionality in Go

Recap: Pure Functions

3

What is a pure function?

- Its return value always the same for the same arguments
- The evaluation has no side effects (no mutation of data outside of the function)
- In other words: After calling a pure function, the rest of the program will be in the same state it was before calling

Examples: exp, sin, cos, max, min.

Advantages

- Reading and understanding is simpler
- Easier to test
- Are less prone to error in general

Recap: Functional Programming – Characteristics

The most prominent characteristics of functional programming are as follows

- Functional programming languages are designed on the concept of mathematical functions that use conditional expressions and recursion to perform computation.
- Functional programming languages don't support flow Controls like loop statements and conditional statements like If-Else and Switch Statements. They directly use the functions and functional calls. Hence, the declarative approach is prevalence rather than the imperative approach.
- Like OOP, functional programming languages can support popular concepts such as Abstraction, Encapsulation, Inheritance, and Polymorphism

Recap: Functional programming offers the following advantages

- Bugs-Free Code

Functional programming does not support state, so there are no side-effect results and we can write error-free codes.

- Efficiency

Functional programs consist of independent units that can **run concurrently**. As a result, such programs can be more efficient.

- Lazy Evaluation

Functional programming supports **lazy evaluation** like Lazy Lists, Lazy Maps, etc.

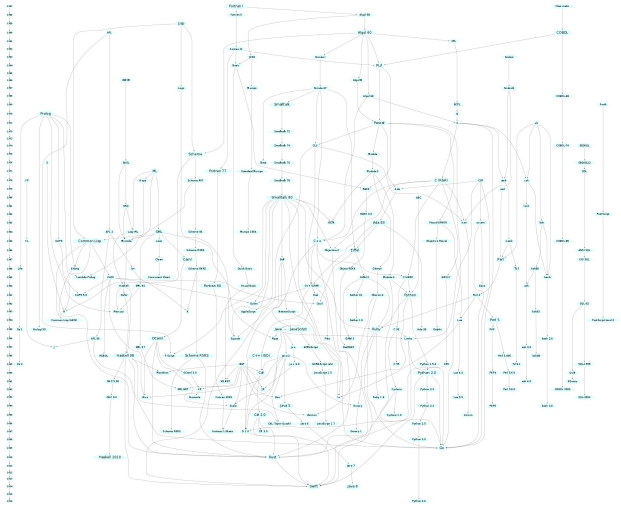
- Distribution

Functional programming supports distributed computing

Haskell

- Haskell was developed by a team led by Professor John Hughes at the University of Glasgow in 1990.
- Haskell is a general-purpose, statically-typed, purely functional programming language with type inference and lazy evaluation
- Designed for teaching, research and industrial applications
- Haskell has pioneered a number of programming language features such as type classes, which enable type-safe operator overloading

img/01-languages.png (img/01-languages.png)

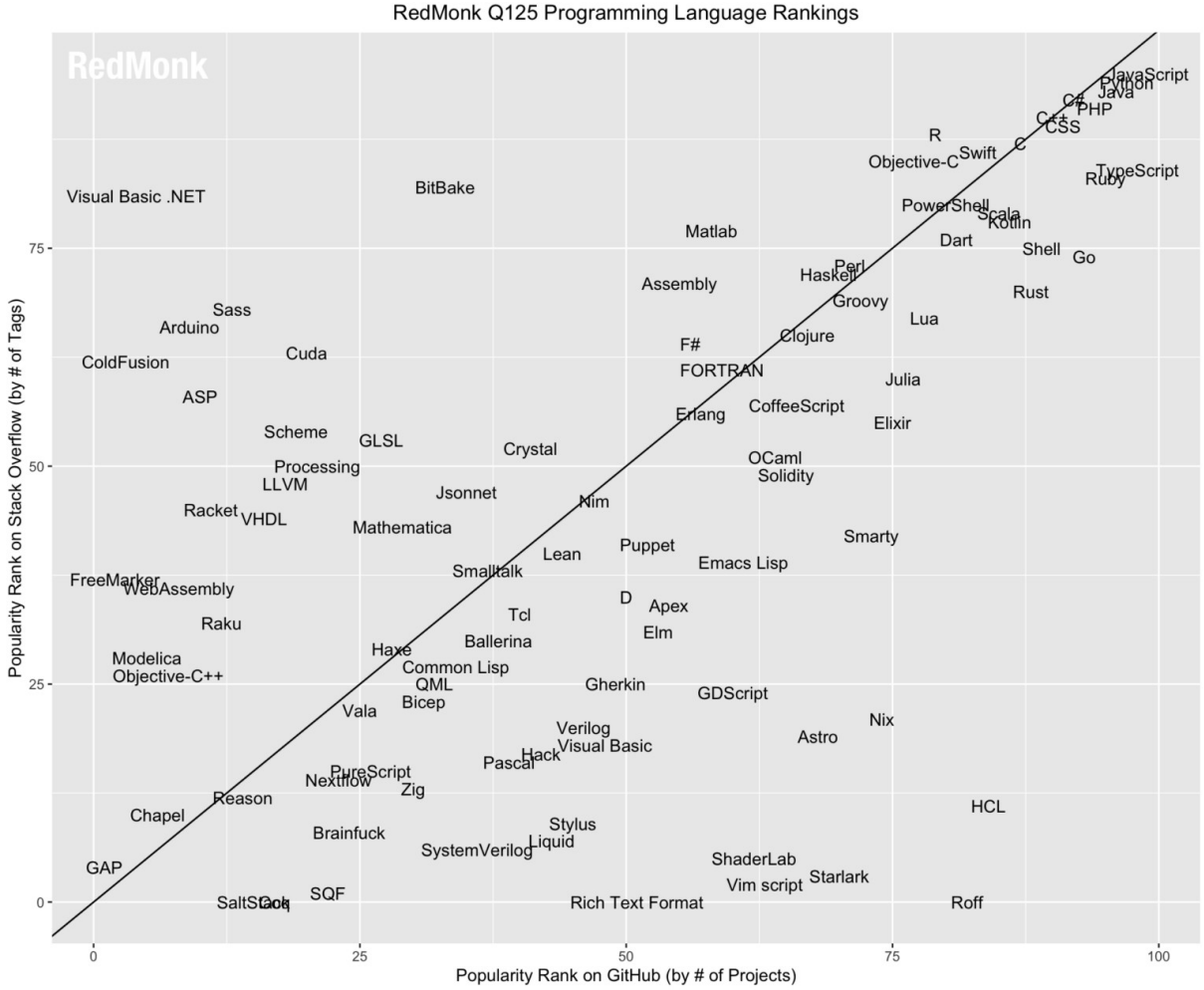


Haskell is used

- in finance for building reliable and mathematically sound trading and risk analysis systems.
- by tech companies such as Facebook and GitHub for tools involving data analysis and code verification
- in academia and research to explore programming language design, type systems, and formal verification
- for developing compilers, interpreters, and domain-specific languages (DSLs) due to its strong abstraction capabilities

Ranking

- 28th most popular programming language by Google searches for tutorials



9

Websites to experiment with Hakell

play.haskell.org/ (https://play.haskell.org/)

www.jdoodle.com/execute-haskell-online (https://www.jdoodle.com/execute-haskell-online)

www.onlinegdb.com/online_haskell_compiler (https://www.onlinegdb.com/online_haskell_compiler)

haskell-wasm.github.io/ghc-in-browser/ (https://haskell-wasm.github.io/ghc-in-browser/)

Online tutorials

learnyouahaskell.com/chapters (http://learnyouahaskell.com/chapters)

learn.hfm.io/ (http://learn.hfm.io/)

hackage.haskell.org/package/CheatSheet-1.11/src/CheatSheet.pdf (https://hackage.haskell.org/package/CheatSheet-1.11/src/CheatSheet.pdf)

Haskell I: Hello world

```
main = print "Hello, World!"
```

1. Define a function with name "main" with no input and output parameters.
Execute the function print, which takes one arbitrary argument. In this case a string

- Printing text doesn't seem to have any side effects

Online editor:

play.haskell.org/ (https://play.haskell.org/)

Haskell: Type System

- Supports typical basic types such as Int, Float, Double, Char, String, Bool, etc.

```
2 + 15                  => 17
40 * 100                => 4900
5 == 5                  => True
5 == 5.0                => True
True && False           => False
not False               => True
"Hello" ++ " World"     => "Hello World"
[1,2,3] ++ [4,5,6]      => [1,2,3,4,5,6]
[1,2,3] !! 1            => 2   // value at index 1
"hey" == ['h','e','y'] => True  -- Strings are just lists of characters
"hey" !! 0              => 'h'
```

12

Haskell: Variables

- Variables are immutable
- Variables are defined locally using the let keyword

```
r = 5 -- globally defined variable
main = do
       let n = r  -- local defined "variable"
       print n
       let m = n + 1
       print m
```

The do keyword is used to define a block of code. This sequence of instructions nearly matches that in any imperative language

Actually a variable is just a function with no arguments

Haskell: Functions

Defines a function **inc** that takes one argument **x** and returns **x** + 1

```
inc x = x + 1   // inc is function name, x is parameter, returns x+1
main = print (inc 1)
```

- Alternative syntax for function calling

```
inc x = x + 1
main = print $ inc 1   // just syntax same as above
```

The **$** operator is used to avoid parentheses. It has the lowest precedence of all operators

Haskell: Partial Applications

Function, which takes multiple arguments, can be partially applied
Here, the function **add** takes two arguments **a** and **b** and returns **a** + **b**

```
add a b = a + b
main = print $ add 1 2
```

- Alternative syntax

```
add a b = a + b
main = do
       let c = add 1 -- partial application
       print $ c 2
```

//compare lambda calculus
//i.e. takes param a and returns function which takes param b

- So this makes sense

```
add a b = a + b
main = print( (add 1) 2)
```

15

Haskell: Type signatures

```
add :: Int -> Int -> Int    // (Int -> (Int -> Int))
add a b = a + b
main = print $ add 1 2
```

- Function signatures are optional, but they provide documentation for other programmers and help the Haskell system to spot type errors
- The function signature can be read as "add" takes two Int arguments and returns an Int
- However with partial application in mind you can read is as "add" takes one Int argument and returns a function that takes one Int argument and returns an Int

```
add :: Int -> (Int -> Int)
```

Haskell: Functional Composition

```
square x = x * x
tripleSquare = square . square . square   //output square -> input "next" square -> input "next" square
main = print $ tripleSquare 2 -- returns 256 //2*2 = 4*4 = 16*16
```

- The . operator is used to compose functions

Haskell: Control Flow

- with if-then-else

```
max a b = if a > b then a else b
main = print $ Main.max 1 2  -- max already exists, so be more specific
```

namespace

- with guards

```
max a b | a > b = a
        | otherwise = b
main = print $ Main.max 1 2 -- max already exists, so be more specific
```

- **guards** are evaluated top to bottom
- **otherwise** is a predefined guard that always evaluates to True

Excercise

- A signum function returns 1 if the number is positive, -1 if the number is negative and 0 if the number is 0.
- Write the signum function using if/else and guards.

19

notes: guards must be inline or it doesnt work, negative numbers need parenthesis

```
sign a | a > 0 = 1
       | a < 0 = -1
       | otherwise = 0
main = print $ sign (-5)
```

Haskell: Recursions

- Using recursion (with the "if then else" expression)

```
factorial n = if n < 2
              then 1
              else n * factorial (n - 1)

main = print $ factorial 10
```

- Using recursion (with pattern matching)

```
factorial 0 = 1
factorial n = n * factorial (n - 1)
main = print (factorial 10)
```

- Pattern matching is evaluated top to bottom
- Pattern matching is more powerful than the "if then else" expression
- Pattern matching is also more efficient, readable and type-safe and more functional

Haskell: Tuples

Haskell uses two fundamental structures for managing several values: lists and tuples. They both work by grouping multiple values into a single combined value.

- How to return multiple values from a function?

```
addsub a b = (a + b, a - b)
main = print $ addsub 1 2
```

- How to access the elements of a tuple?

```
add a b = (a + b, a - b)
main = do
      let (x, y) = add 1 2
      print x; print y
      print $ fst (add 1 2) -- first element
      print $ snd (add 1 2) -- second element
```

- can be used also as key-value pair or as a list or as 2D-coordinates
- In contrary to lists, tuples have a fixed number of elements and each element can have a different type

Haskell: Lists

- Lists are a fundamental data structure in Haskell
- Lists are homogenous, i.e. all elements of a list must have the same type
- Lists are immutable

```
main = do
     print [1,2,3]
     print (1:2:3:[])    -- same as [1,2,3]
     print [1 .. 3]      -- same as [1,2,3]
     print ([1,2,3] ++ [4,5,6])
     print $ [1,2,3] !! 1 -- access element at index 1
     print $ head [1,2,3] -- first element
     print $ tail [1,2,3] -- all elements except the first one
     print $ last [1,2,3] -- last element
     print $ init [1,2,3] -- all elements except the last one
```

22

Haskell: Functional usage of lists

```
myhead (x:xs) = x
mytail (x:xs) = xs
main = do
       print $ myhead [1,2,3]
       print $ mytail [1,2,3]
```

(x:xs) is a pattern that matches a list with at least one element

- x is the first element of the list
- xs is the rest of the list

Haskell: Functional usage of lists

- How to reverse a list?

```
reverse [] = []
reverse (x:xs) = Main.reverse xs ++ [x]
main = print $ Main.reverse [1,2,3]
```

//partial application, recursively calls reverse with the rest of the list until it has iterated through (xs is empty), then returns empty list and appends the first element of each rest in reverse

- How to check if a list is a palindrome?

```
isPalindrome [] = True
isPalindrome [x] = True
isPalindrome (x:xs) = x == (last xs) && isPalindrome (init xs)
main = print $ isPalindrome [1,2,3,2,1]
```

24

"iterates" through the array and through each recursion moves one entry further in

Haskell: Functional Composition with Lists

```
import Data.List
revsort = (reverse . sort)
main = print $ revsort [2,8,7,10,1,9,5,3,4,6]
```

1. Sort takes a list and returns a sorted list. The result of sort is pipelined to reverse
2. prints the result of revsort for the given table

25

Haskell: Lazy evaluation

```
osc :: [Integer]          defines osc as returning an array
osc = 0 : 1 : osc         returns array with entries {0,1} + osc, recursively appending 0:1
main = print (osc!!10)
```

1. Define a function osc, which returns a list of integers of unknown size
2. Define a function osc, which returns oscillating numbers 0,1,0,1,0,1. There is no break condition in this list!
3. Execute the function fibs which actually should return an infinite list. But is not evaluated yet.

Take the 10th element out of the list. Print the result

This example explains the concept of lazy evaluation.
The list osc is not evaluated until it is needed.
The function osc is called only once, but the list osc is evaluated many times.
The list osc is evaluated only as much as needed.

Higher Order Functions

- A function that takes another function as an argument or returns a function as a result is called a higher order function
- map takes a function and a list as arguments and returns a list

```
map :: (a -> b) -> [a] -> [b]
map f [] = []
map f (x:xs) = f x : map f xs

main = print $ map (+1) [1,2,3]
```

(+1) implicitly declares function add a = a+1

- Can also be used with partial application

```
add x y = x + y

map :: (a -> b) -> [a] -> [b]
map f [] = []
map f (x:xs) = f x : Main.map f xs

main = print $ Main.map (add 1) [1,2,3]
```

27

Finally: The quicksort algorithm by using the filter function

With the filter function

```
main = print $ filter (<3) [4,3,2,1,2,3,4]    -- returns [2,1,2]
```

we can finally define the famous quicksort algorithm

```
quickSort []     = []
quickSort (x:xs) = quickSort (filter (<x) xs)
                   ++ [x] ++
                   quickSort (filter (>=x) xs)
main = print $ quickSort [2,8,7,10,1,9,5,3,4,6]
```

Communication with the outside world

Also Haskell have to perform IO

```
import Data.Time.Clock
import Control.Concurrent

main :: IO ()   -- return type is IO ().
main = do
  x <- getCurrentTime;
  print x;
  threadDelay 1000000;
  x <- getCurrentTime;
  print x
```

The <- sign is used to assign the result of a non-pure function to a variable

IO Sequencing with do Notation and return in Haskell

```
import Data.Time.Clock
import Data.Time
import Control.Concurrent

waitOneSecond :: IO NominalDiffTime
waitOneSecond = do
  t1 <- getCurrentTime;
  print t1;
  threadDelay 1000000;
  t2 <- getCurrentTime;
  print t2
  return (diffUTCTime t2 t1)

main :: IO ()
main = do
    result <- waitOneSecond
    print $ "waited " ++ show result
```

30

Exercise 6

github.com/s-macke/concepts-of-programming-languages/blob/master/docs/exercises/Exercise6.md (https://github.com/s-macke/concepts-of-programming-languages/blob/master/docs/exercises/Exercise6.md)

31

Thank you

Tags: go, programming, master (#ZgotmplZ)

Sebastian Macke
Rosenheim Technical University
Sebastian.Macke@th-rosenheim.de (mailto:Sebastian.Macke@th-rosenheim.de)
https://www.qaware.de (https://www.qaware.de)